Project Description and Grading Rubric: BDH Reproducibility Challenge

Jimeng Sun

***Abstract*—CSE6250 Big Data for Health Informatics is a graduate-level course focusing on practical big-data technology and machine learning for healthcare applications. A major component of the course is a team reproducibility project in which students study a recent healthcare ML/DL paper, reproduce a meaningful part of its computational pipeline, evaluate the reproducibility of the published claims, and contribute reusable software to the PyHealth ecosystem. Generative AI assistants such as ChatGPT, Claude, and Gemini may be used as part of the workflow, but students are expected to validate AI-generated content and document how these tools were used. This document defines the project expectations, milestones, submission requirements, and grading rubric for the course project.**

***Index Terms*—Machine learning, deep learning, healthcare, reproducibility, PyHealth, generative AI.**

I. Project Objective

Each team will select a paper from the course paper pool and attempt to reproduce a meaningful part of the work. The goal is not simply to obtain the same final number reported in the paper. Instead, the project should demonstrate that your team can:

- understand the scientific problem, methodology, data, and evaluation protocol;
- implement a useful part of the paper's pipeline in PyHealth;
- write clean, documented, tested, and reusable code;
- compare your reproduced results with the claims in the original paper;
- perform at least one meaningful ablation or extension; and
- explain what was reproducible, what was difficult, and why.

A reproduction may be positive (the main finding can be supported) or negative (the result cannot be reproduced because of missing data, code, implementation details, computational limits, or other barriers). Both outcomes are acceptable. Grading emphasizes a good-faith reproducibility effort, sound engineering practice, clear evidence, and thoughtful analysis.

II. Paper Selection

Choose a paper from the provided paper pool that is both interesting to your team and realistically reproducible within the semester. Before committing to a paper, consider the following:

- Is the research question clear and relevant to healthcare ML/DL?
- Can your team access the required data, or a suitable public/synthetic substitute?
- Are the computational requirements feasible with the resources available to you?
- Is there a meaningful component that can be integrated into PyHealth?
- Is there an implementable ablation or extension that is not already reported in the paper?

You may use the original authors' code as a reference when it is publicly available, but your submitted contribution must demonstrate your own understanding and implementation. Directly copying the original repository without meaningful reimplementation, testing, documentation, and integration is not sufficient.

III. Teams

The default CSE6250 policy remains a **two-student team**. Individual projects are not permitted unless explicitly approved by the course staff. Both members normally receive the same project score; substantial imbalance in contribution may result in an individual adjustment.

To avoid excessive duplication, each paper may be selected by at most **two teams**, on a first-come, first-served basis, unless course staff announces a different limit.

IV. Project Milestones

No late submission is guaranteed for any project component. The project is graded on a **100-point scale**; its contribution to the overall course grade follows the course syllabus.

Due Date	Task
Sep 28 2026	Team formation and paper selection (3 pt) + consent/acknowledgment (2 pt)
Oct 05 2026	Project proposal (20 pts)
Nov 30 2026	Final project: PyHealth contribution + AI-use form + video (75 pts + 10 extra-credit pts)

V. Grading Overview

Component	Points
Team formation and paper selection	3
Consent / class-wide AI-study acknowledgment	2
Project proposal PDF	15
Generative AI Assistant form for proposal	5
PyHealth contribution / pull request	50
Final-project Generative AI form	15
Video presentation	10
Total	**100**
Full-pipeline extra credit	+10

Important change for Fall 2026: the final project is centered on the **PyHealth contribution, final Generative AI form, and video presentation**. There is **no separate final report**. The proposal remains the only required project write-up.

VI. Project Requirements

A. Team Formation and Paper Selection (3 pts)

Deadline: Sep 28 2026

Register your team and paper selection using the course signup sheet:

Team Registration & Paper Selection Sheet

1) Registration and Submission Requirements: Each team should:

a) Register both team members using their names, GT usernames, and GT email addresses.
b) Review the paper pool and select candidate paper(s) after checking data accessibility and computational feasibility.
c) Record the selected paper index, title, and authors in the signup sheet.
d) Submit a **PDF no longer than 1 page** to the corresponding Gradescope assignment. The PDF may use any format but must contain:
   - names and GT usernames of both team members;
   - indices, titles, and authors of the selected papers.
e) The Gradescope submission must be made **at the team level**. Follow this team submission instruction and make sure both team members are included in the submission.

Once the selection deadline has passed, team membership and paper assignments are considered final unless a change is approved by the course staff.

B. Consent / Class-Wide Generative AI Study Acknowledgment (2 pts)

The course may use de-identified project artifacts (for example, prompts, generated code, and feedback) to study how generative AI supports healthcare-AI education and reproducibility work. Every student must submit the consent/decline acknowledgment on Gradescope for completion credit.

Participation in the research study is voluntary and does not affect the course grade. The **2 points are awarded for completing the acknowledgment**, regardless of whether the student agrees or declines participation. Follow the study language and consent instructions posted by the course staff.

C. Project Proposal (20 pts total)

Deadline: Oct 05 2026

Only **one proposal PDF per team** is required on Gradescope. In addition, **each student** must complete the Generative AI Assistant Form.

The project proposal consists of two graded components:

- **Proposal PDF: 15 pts**
- **Generative AI Assistant Form: 5 pts**

The proposal should demonstrate that the team understands the selected paper, has verified that a meaningful portion of the work is reproducible, has access to appropriate data and computational resources, and has developed a realistic plan for a PyHealth contribution.

*1) **Proposal PDF (15 pts):*** Submit a **2-page main-content PDF** with unlimited references. An AAAI LaTeX template is recommended:

AAAI Author Kit on Overleaf

Use your own words. You may use an LLM to assist with reading, planning, or drafting, but the final proposal must reflect your team's validated understanding rather than copied AI output.

The Proposal PDF will be graded according to the following rubric:

a) Introduction (3 pts)
   - Problem Statement (2 pts): Clearly state the healthcare problem addressed by the paper and the specific reproduction target of your project.
   - Original Paper Citation (1 pt): Properly cite the original paper.
b) Methodology (6 pts)
   - Specific Approach (2 pts): Describe the models, techniques, pipeline components, and evaluation metrics used in the original work. Explain how the relevant components will be implemented in PyHealth.
   - Novelty, Relevance, and Hypotheses (2 pts): Explain the paper's claimed contribution, why the proposed method is expected to improve upon relevant baselines, and which hypotheses or claims your team plans to evaluate.
   - Planned Ablation or Extension (2 pts): Propose at least one meaningful and implementable analysis, ablation, or extension that is not simply an experiment already reported in the original paper.
c) Data Access and Implementation Details (5 pts)
   - Data and Resource Accessibility (2 pts): Describe the dataset, current data-access status, required computational/model resources, and whether the authors provide a public codebase.
   - PyHealth Implementation Plan (2 pts): Explain how the dataset, task, model, or other selected contribution will be represented and implemented in PyHealth.
   - Use of Original Code (1 pt): Clearly state whether the original authors' code is available and, if so, how it will be used as a reference rather than directly copied.
d) Formatting and Page Limit (1 pt)
   - The proposal main content must not exceed **2 pages**.
   - References are unlimited and do not count toward the 2-page main-content limit.

*2) **Generative AI Assistant Form for Proposal (5 pts):*** Each student should complete the proposal-stage form **while working on the proposal**, rather than attempting to reconstruct the interaction at the end:

Generative AI Assistant Form – Proposal Stage

The form records items such as:

- the generative AI model(s) used;

- prompts used to understand the problem, methodology, and novelty;
- relevant AI-generated responses;
- your validation of the correctness and usefulness of those responses; and
- your analysis of how prompt design affected the quality of the assistance.

The form is graded primarily for thoughtful completion. Do **not** paste raw prompts and AI responses into the 2-page proposal unless specifically requested. Submit those materials through the designated form.

VII. Final Project (75 pts + 10 Extra Credit)

A. *Final Project Overview*

Deadline: Nov 30 2026

The final project contains exactly three graded components:

- PyHealth Contribution / Pull Request: 50 pts + up to 10 extra-credit pts
- Final-Project Generative AI Form: 15 pts
- Video Presentation: 10 pts

There is **no final report**. You do not need to reproduce the selected paper in its entirety. A high-quality reproduction of a meaningful dataset, task, model, or pipeline component is sufficient when it is directly connected to the selected paper.

Ablations and extensions must be **implemented and evaluated**, not merely proposed. If computational resources are limited, teams may use a smaller subset of the data, provided that the limitation is clearly documented.

B. *PyHealth Contribution / Pull Request (50 pts + 10 Extra Credit)*

The PyHealth contribution is the primary technical component of the final project. Each team should reproduce a meaningful portion of the selected paper through a contribution that is suitable for integration into the PyHealth ecosystem.

*1) **Submission Requirements**:* Each team submits **one pull request** to the PyHealth GitHub repository and submits the PR link on Gradescope.

The pull request must satisfy the following requirements:

- Target the current `main` branch.
- Rebase or update against remote `main` before opening the PR.
- Enable "Allow edits by maintainers."
- Include a link to the original paper in the PR description.
- Include an ablation/example script under `examples/` using the naming pattern `dataset_task_model.py`.
- Add all group members to the Gradescope submission.

The PyHealth contribution guide and example are available here:

PyHealth – How to Contribute & Excellent Template

If the pull request is accepted by PyHealth before the grading deadline, the team receives full credit for the **50-point PyHealth contribution component**. The Generative AI Form and Video Presentation are still required.

*2) **Valid Contribution Options**:* Complete reproduction is not always possible because of unavailable data, missing implementation details, computational limitations, or other constraints. Teams may therefore choose one of the following contribution scopes:

a) Dataset + Task
- Implement a dataset and its associated prediction or processing task.
- Provide comprehensive tests for both the dataset and task.
- Provide an ablation/example script showing meaningful task or feature variations.
- Example: ChestXray14 and CosmicDataset

b) Model
- Implement a model from the selected paper that provides a meaningful contribution to PyHealth.
- Provide comprehensive model tests.
- Provide an ablation/example script evaluating hyperparameter variations or another meaningful model analysis.
- Example TFM-Tokenizer

c) Standalone Task
- Implement a new task for an existing PyHealth-supported dataset.
- Provide comprehensive task tests.
- Provide an example or ablation demonstrating different task configurations.

d) Full Pipeline
- Implement an end-to-end contribution containing dataset + task + model.
- A substantial model + task subset may also qualify when appropriate.
- The contribution should be novel relative to the current PyHealth implementation and directly grounded in the selected paper.
- Include complete documentation, testing, a runnable example, and a pipeline-level ablation.
- A qualifying full-pipeline contribution may receive up to **10 extra-credit points**.

*3) **PR Quality and Completeness**:* The PyHealth pull request is evaluated using a total of **50 points**, distributed across the following five categories:

$$\begin{aligned}\text{Total} = {} & 20 \text{ pts (Core Implementation)} \\ & + 12 \text{ pts (Required File Updates)} \\ & + 10 \text{ pts (Test Cases)} \\ & + 5 \text{ pts (Ablation Study / Example Usage)} \\ & + 3 \text{ pts (Pull Request Formatting)} \\ = {} & 50 \text{ pts.}\end{aligned}$$

The detailed rubric consists of the following five categories.

*a) **Core Implementation (20 pts)**:* The detailed implementation requirements depend on the selected contribution type.

For Dataset Contribution (must include an associated task):

- Code Implementation (12 pts)
  - Properly inherits from `BaseDataset` (2 pts).
  - Implements required data loading, parsing, and configuration support (5 pts).
  - Follows established PyHealth implementation patterns (3 pts).
  - Includes appropriate error handling and validation (2 pts).
- Documentation (5 pts)
  - Comprehensive Google-style docstrings (2 pts).
  - Proper type hints for all methods (2 pts).
  - Clear high-level description and usage example (1 pt).
- Code Quality (3 pts)
  - Follows PEP8 conventions and the 88-character line-length guideline (1 pt).
  - Uses appropriate naming conventions (1 pt).
  - Code is well structured and readable (1 pt).

For Model Contribution:

- Code Implementation (12 pts)
  - Properly inherits from `BaseModel` (2 pts).
  - Implements the required abstract methods correctly (5 pts).
  - Provides a clear forward-pass implementation (3 pts).
  - Includes appropriate initialization and configuration methods (2 pts).
- Documentation (5 pts)
  - Comprehensive Google-style docstrings (2 pts).
  - Proper type hints for all methods (2 pts).
  - Clear high-level description and usage example (1 pt).
- Code Quality (3 pts)
  - Follows PEP8 conventions and the 88-character line-length guideline (1 pt).
  - Uses appropriate naming conventions (1 pt).
  - Code is well structured and readable (1 pt).

For Standalone Task Contribution:

- Code Implementation (12 pts)
  - Properly integrates with the appropriate PyHealth task infrastructure (2 pts).
  - Defines clear task inputs and outputs (5 pts).
  - Implements feature extraction and label generation correctly (3 pts).
  - Follows established PyHealth task patterns (2 pts).
- Documentation (5 pts)
  - Comprehensive Google-style docstrings (2 pts).
  - Proper type hints for all methods (2 pts).
  - Clear high-level description and usage example (1 pt).
- Code Quality (3 pts)
  - Follows PEP8 conventions and the 88-character line-length guideline (1 pt).
  - Uses appropriate naming conventions (1 pt).
  - Code is well structured and readable (1 pt).

*b) **Required File Updates (12 pts)**:* The pull request must include all files required for the selected contribution type. The expected files are listed below.

- Core Implementation File(s) (3 pts)
  - Dataset + Task contribution:
    - Dataset implementation: `pyhealth/datasets/new_dataset.py`
    - Associated task implementation: `pyhealth/tasks/new_task.py`
    - Dataset configuration: `pyhealth/datasets/configs/dataset.yaml`
  - Model contribution:
    - Model implementation: `pyhealth/models/new_model.py`
  - Standalone Task contribution:
    - Task implementation: `pyhealth/tasks/new_task.py`
- Documentation RST File(s) (3 pts)
  The contribution must include the corresponding API documentation file(s).
  - Dataset + Task contribution:
    Both of the following documentation files are required:
    - Dataset API documentation: `docs/api/datasets/pyhealth.datasets.new_dataset.rst`
    - Task API documentation: `docs/api/tasks/pyhealth.tasks.new_task.rst`
  - Model contribution:
    - Model API documentation: `docs/api/models/pyhealth.models.new_model.rst`
  - Standalone Task contribution:
    - Task API documentation: `docs/api/tasks/pyhealth.tasks.new_task.rst`

  Each documentation file should expose the new component through the PyHealth documentation system and provide sufficient information for users to understand and locate the implementation.
- Documentation Index / Table-of-Contents Updates (2 pts)
  The new contribution must also be added to the appropriate documentation index or table-of-contents file(s), so that the new API documentation appears in the generated PyHealth documentation.
  Examples include:
  - Dataset contributions: `docs/api/datasets.rst`
  - Task contributions: `docs/api/tasks.rst`
  - Model contributions: `docs/api/models.rst`

  Update all relevant index files when a contribution contains more than one component. For example, a Dataset + Task contribution should update both the dataset and task documentation indices.
- Ablation / Example Script (1 pt)
  Include a runnable example or ablation script under the `examples/` directory using the naming convention

`examples/{dataset}_{task_name}_{model}.py`

or a clearly equivalent descriptive filename when the standard naming pattern is not applicable.
The script should:

- demonstrate how the contributed component is used;
- include the required ablation study or example experiment;
- be directly related to the selected paper and reproduction target; and
- contain sufficient comments or documentation to explain the experimental setup.

- Test Case File(s) (3 pts)
  Appropriate unit tests must be included for the selected contribution type.
  - Dataset + Task contribution:
    - Include tests for both the dataset implementation **and** the associated task.
    - Dataset tests should normally use only **2–5 synthetic patients**.
  - Model contribution:
    - Include model-specific tests.
    - Use minimal synthetic tensors sufficient to test the model behavior.
  - Standalone Task contribution:
    - Include tests for the task using an existing PyHealth-supported dataset interface or an appropriate synthetic representation.

  All tests must satisfy the following requirements:
  - Use **synthetic or pseudo data**; do not use real protected datasets or full demo datasets directly in unit tests.
  - Tests should be lightweight and execute quickly, ideally in milliseconds per individual test.
  - Dataset tests should use only the minimum number of synthetic patients required to verify functionality.
  - Model tests should use minimal synthetic tensors.
  - Tests should be deterministic and suitable for automated continuous-integration testing.

  *c) **Test Cases (10 pts)**:*

- Fast and Performant Tests (5 pts)
  - Uses small synthetic or pseudo data rather than real datasets or full demo datasets such as MIMIC demo data (2 pts).
  - Tests execute quickly, ideally in milliseconds per individual test and within seconds for the entire test suite (2 pts).
  - Uses temporary directories and performs proper cleanup when file-system operations are required (1 pt).
- Comprehensive Coverage (5 pts)
  The following requirements are evaluated according to the applicable contribution type:
  - Dataset contributions: test data loading, patient parsing, event parsing, data integrity, and associated task functionality (5 pts).
  - Model contributions: test model instantiation, forward pass, output shapes, and gradient computation (5 pts).
  - Standalone Task contributions: test sample processing, label generation, feature extraction, and relevant edge cases (5 pts).

Tests should be deterministic, lightweight, and suitable for automated continuous-integration testing. Dataset tests should generally use only a small number of synthetic patients, while model tests should use minimal synthetic tensors.

*d) **Ablation Study / Example Usage (5 pts)**:* The ablation study or example usage must be demonstrated in the project's example script, normally following the naming pattern:

`examples/{dataset}_{task_name}_{model}.py`

The experiment must be meaningfully connected to the selected paper and its reproduction target.

For Model Contributions:

- Vary meaningful model hyperparameters or configurations, such as learning rate, hidden dimensions, dropout, or other relevant settings, using an existing PyHealth dataset (2 pts).
- Compare model performance across the tested configurations using appropriate evaluation metrics (2 pts).
- Clearly document the experimental setup, configurations, and findings (1 pt).

For Dataset / Task Contributions:

- Vary meaningful dataset features or task configurations, such as prediction windows, feature sets, label definitions, or other task-specific settings (2 pts).
- Evaluate how these variations affect model performance using an existing PyHealth model (2 pts).
- Clearly document the experimental setup, configurations, and findings (1 pt).

The ablation/example must satisfy the following requirements:

- It must be runnable with synthetic or demo data.
- It must include clear comments explaining the purpose and setup of the ablation study.
- Experimental results must be documented in a docstring, accompanying Markdown file, or other clearly associated documentation.

Notebook-based examples (`.ipynb`) are allowed. However, standard Python scripts are preferred because notebooks may run considerably more slowly in the current evaluation environment.

*e) **Pull Request Formatting (3 pts)**:* The pull request description must include:

- contributor names and GT usernames/emails (0.5 pt);
- contribution type: Dataset + Task, Model, Standalone Task, or Full Pipeline (0.5 pt);
- a link to the original paper being reproduced (0.5 pt);
- a clear high-level description of the implementation and its relationship to the selected paper (0.5 pt);
- a file guide identifying the primary files that reviewers should inspect (0.5 pt); and
- formatting consistent with the example pull-request format in the PyHealth contribution guide (0.5 pt).

Paper-replication relevance requirement: The pull request must directly support the reproduction of the team's

selected paper. If the submitted PR is unrelated to the selected paper's replication target, **3 raw points will be deducted**.

*4) **Extra Credit: Full Pipeline Replication (+10 pts)**:* Up to **10 extra-credit points** may be awarded for a complete end-to-end reproduction contribution that goes beyond the requirements of the standard 50-point PyHealth contribution.

To qualify for full extra credit, the contribution should include all of the following:

- a complete **Dataset + Task + Model** pipeline, with each component representing a substantive contribution related to the selected paper;
- complete documentation and appropriate testing for all implemented components;
- a working example, preferably under the `examples/` directory, demonstrating the full pipeline:

  data loading $\rightarrow$ task processing $\rightarrow$ model training $\rightarrow$ evaluation;

- a comprehensive ablation study or extension evaluated across the pipeline;
- successful end-to-end execution of the implemented pipeline; and
- reproducibility using synthetic or demo data so that the contribution can be reviewed without requiring access to protected datasets.

Partial extra credit may be awarded when a submission demonstrates substantial progress toward a full-pipeline contribution but does not satisfy every requirement above.

*5) **Common Deductions**:* The following deductions may apply when required elements of the PyHealth contribution are missing or substantially incomplete:

- Dataset contribution without an associated task: up to **-15 pts**;
- Missing ablation study or example script: up to **-5 pts**;
- Missing link to the original paper in the PR description: **-1 pt**;
- Use of real or protected datasets directly in unit tests: up to **-5 pts**;
- Incomplete or insufficient documentation: typically **-3 to -5 pts**;
- Unnecessarily slow unit tests (for example, individual tests taking more than approximately one second without clear justification): typically **-3 pts**;
- Missing required implementation, documentation, or index-file updates: typically **-2 to -3 pts per missing item**; and
- Poor code quality, organization, or readability: typically **-2 to -4 pts**.

These deductions are intended to address missing or deficient requirements and are not additional rubric categories. Course staff may adjust the deduction when the severity of the issue differs from the typical cases listed above.

C. *Final-Project Generative AI Form (15 pts)*

Each student must complete the Final-Project Generative AI Form individually:

Generative AI Assistant Form – Final Project

The form is graded primarily for thoughtful and complete submission. Students should complete the form throughout the project rather than attempting to reconstruct their AI interactions only at the end.

The form may include questions related to:

- AI assistance with data preprocessing;
- model implementation;
- novelty and methodological analysis;
- development or validation of evaluation code;
- extension or ablation brainstorming;
- implementation of extensions or ablations;
- validation of AI-generated outputs; and
- insights and analysis on how prompt design affected the usefulness and correctness of AI assistance.

Students should use generative AI as an assistant rather than as a substitute for technical judgment. AI-generated outputs should be independently checked against the selected paper, documentation, code behavior, experimental results, or other appropriate technical evidence.

Course staff may compare submitted form responses with the actual code, experiments, and project artifacts. Unsupported, unvalidated, or direct copy/paste responses may lose credit.

Note: The Final-Project Generative AI Form is an **individual submission**, even though the technical project and PyHealth contribution are completed at the team level.

D. *Video Presentation (10 pts)*

Each team must prepare a recorded presentation that provides a concise walkthrough of the selected paper, the reproduction effort, the obtained results, and the implemented extension or ablation.

Please prepare presentation slides and upload the recorded video to a publicly accessible or course-accessible platform such as YouTube, OneDrive, Google Drive, or another platform that can be accessed by the course staff without requesting additional permission.

The submitted video link must remain accessible throughout the grading period.

The presentation should be **4–7 minutes long**. Content presented after the 7-minute mark is not guaranteed to be considered for grading.

The presentation is graded according to the following rubric:

a) Presentation Length and Organization (2 pts)
   - The presentation is between **4 and 7 minutes**.
   - The presentation is clearly organized and appropriately paced.
   - Slides and narration should allow the grader to follow the project without needing to read the original paper first.

b) General Problem (2 pts)
   - Clearly explain the healthcare problem addressed by the original paper.
   - Briefly explain why the problem is important or relevant.

c) Specific Approach (2 pts)

- Clearly explain the main methodology, model, dataset/task pipeline, or technical approach used in the original paper.
- Focus on the components most relevant to your reproduction.

d) Reproduction Attempt and Results (2 pts)
- Clearly describe what portion of the original work your team attempted to reproduce.
- Show the results obtained from your implementation.
- Compare your results with the corresponding results reported in the original paper.
- Briefly explain why your results match, outperform, underperform, or otherwise differ from those reported in the paper.

e) Extensions / Ablations (2 pts)
- Explain the implemented extension or ablation.
- Explain why the experiment was selected.
- Present the resulting findings and briefly discuss what was learned from the experiment.

Use readable slides, appropriately sized text, and clearly labeled figures and tables. The presentation should provide enough information for the grader to understand the original work, your implementation, the evidence supporting your reproducibility assessment, and the additional extension or ablation performed by your team.

VIII. Generative AI Usage Expectations

Generative AI tools are permitted and intentionally incorporated into the project workflow. However, students remain responsible for the correctness of all submitted work.

Throughout the project, keep track of:

- which AI assistant(s) you used;
- important prompts used to understand the paper or generate/debug code;
- the initial AI output and whether it was correct, relevant, and helpful;
- how you validated the output against the paper, documentation, code behavior, or experiments;
- how prompts were revised when initial responses were incomplete or incorrect; and
- your own analysis of what made the AI assistance effective or ineffective.

Do not submit unvalidated AI-generated code or prose as if it were independently verified work.

IX. MIMIC / PhysioNet Access (When Applicable)

Teams selecting papers that require MIMIC data should begin credentialing early. In general, PhysioNet requires account registration, the applicable data-use agreement, and required CITI training. Course-specific event/credentialing links will be announced separately by CSE6250 staff.

Useful public resources:

- CITI training instructions: Link
- MIMIC-IV demo data for development while waiting for full access: Link
- MIMIC dataset application instruction: Instructions

Do not place protected patient data in a public repository or in unit tests. Use synthetic/pseudo data for automated tests.

X. Submission and Link Summary

Item	Submission / Link
Team/paper signup	Course signup sheet
Gradescope	Gradescope
Proposal AI form	Proposal-stage form
Final AI form	Final-project form
PyHealth guide	Contribution guide
PyHealth repository	GitHub repository